TP 1 à 4: Serveur d'Authentification

Travail individuel

- Construire progressivement une même application API REST sécurisée en **Java** avec **Spring Boot** et MySQL tout en utilisant **SonarCloud** et le **versionning**
- Comprendre les risques liés à une authentification
- Passer d'une authentification simple qui fonctionne à une authentification robuste de **niveau industriel**
- Mettre en place une **chaîne CI/CD** pour automatiser les contrôles de qualité
- Produire une API REST sécurisée, documentée et industrialisée conforme aux attentes du parcours CDWFS

GitHub sonarcloud spring boot

TP1 – Authentification Dangereuse

Objectif:

- Mettre en place une authentification **volontairement dangereuse**, pour servir de base aux TP suivants.
- Comprendre pourquoi un système peut fonctionner et rester inacceptable en production.
- Durée estimée : 10 heures (**1 semaine**)
- Version Git attendue : **tag v1-tp1**

Objectif Général

Construire progressivement:

- Un **serveur d'authentification** avec API REST sécurisée en Java avec **Spring Boot** et MySQL
- Comprendre les différentes méthodes d'authentification et leurs limites de sécurité
- Passer d'une authentification simple qui fonctionne à une authentification robuste de niveau industriel
- Apprendre à sécuriser une API REST selon les bonnes pratiques professionnelles
- Mettre en place une API documentée et testée
- Intégrer les notions de qualité logicielle et d'industrialisation
- Comprendre les risques liés à une authentification mal conçue
- Mettre en œuvre une architecture propre et maintenable
- Apprendre à tester et valider la sécurité d'une API
- Mettre en place une chaîne CI/CD pour automatiser les contrôles de qualité

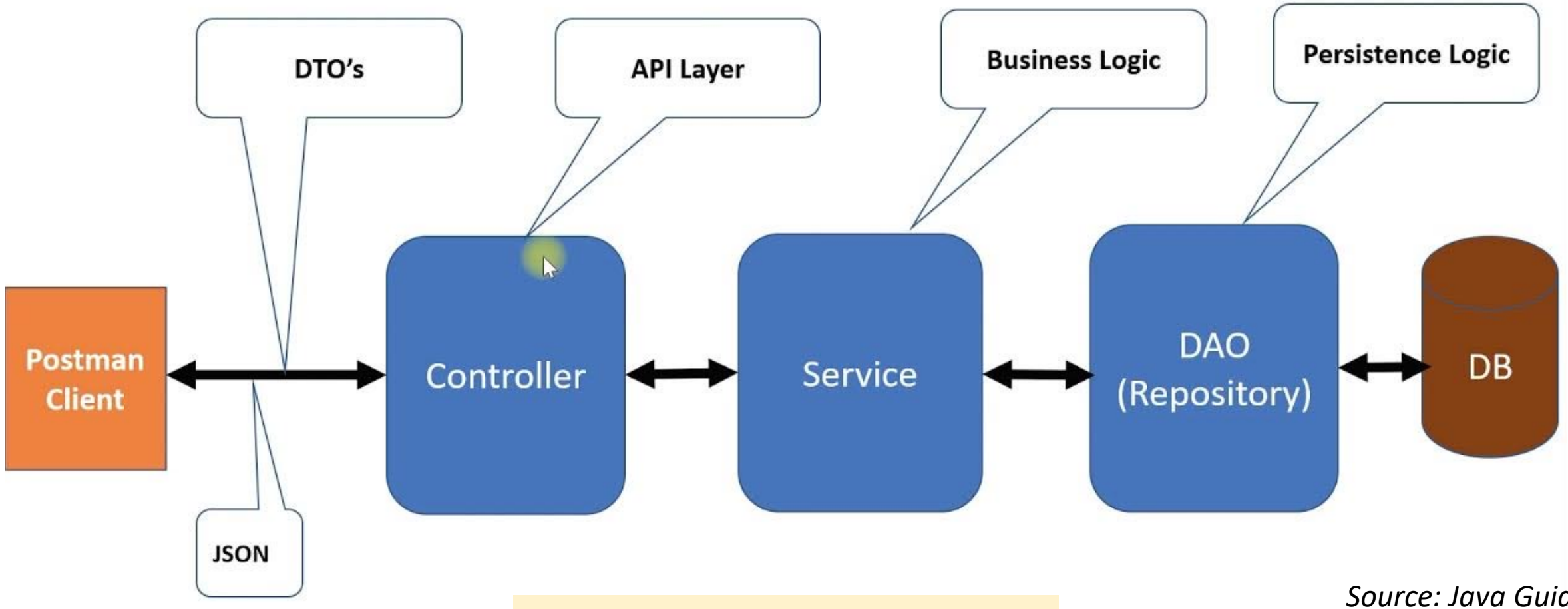
Objectifs pédagogiques progressifs

- Implémenter une première authentification simple fonctionnelle
- Comprendre pourquoi une authentification simple peut être vulnérable
- Améliorer progressivement la sécurité de l'authentification
- Mettre en place des mécanismes d'authentification standard
- Sécuriser l'accès aux endpoints REST
- Mettre en place une gestion des utilisateurs et des rôles
- Documenter l'API avec OpenAPI ou Swagger
- Mettre en place des tests unitaires et des tests d'API
- Automatiser les tests avec CI/CD
- Atteindre un niveau proche des pratiques industrielles

Objectif final attendu

- Réaliser une API REST professionnelle sécurisée avec
- Authentification robuste, documentation complète,
- Tests automatisés et intégration continue.

Architecture d'Application avec Spring Boot



Source: Java Guide

Objectif principal du TP:

- Installer, configurer
- Apprendre et utiliser Spring Boot

Architecture imposée

Technos:

- Client lourd Java (Swing, JavaFX ou WindowBuilder).
- Back-end Spring Boot API REST et base MySQL.

Fonctionnalités attendues

- Inscription: le client permet de créer un compte avec email et mot de passe.
- Règles mot de passe volontairement faibles :
- Minimum 4 caractères, aucune autre règle.

Côté serveur :

- Email unique.
- Mot de passe stocké en clair en base.
- Endpoint POST /api/auth/login.
- Vérifier email et mot de passe.
- Retourner succès ou échec.

Compte de test obligatoire :

- toto@example.com
- pwd1234
- Route protégée
- Endpoint GET /api/me.
- Accessible uniquement si l'utilisateur est authentifié.
-

Vous pouvez utiliser :

- Une session HTTP simple, ou
- Un token basique généré côté serveur et stocké en base.

Remarques:

- Aucune logique de validation de sécurité côté client uniquement.
- Toute validation doit être dupliquée côté serveur.

Modèle de données minimal

Table users :

- Id, email, password_clear, created_at

Structure Spring recommandée

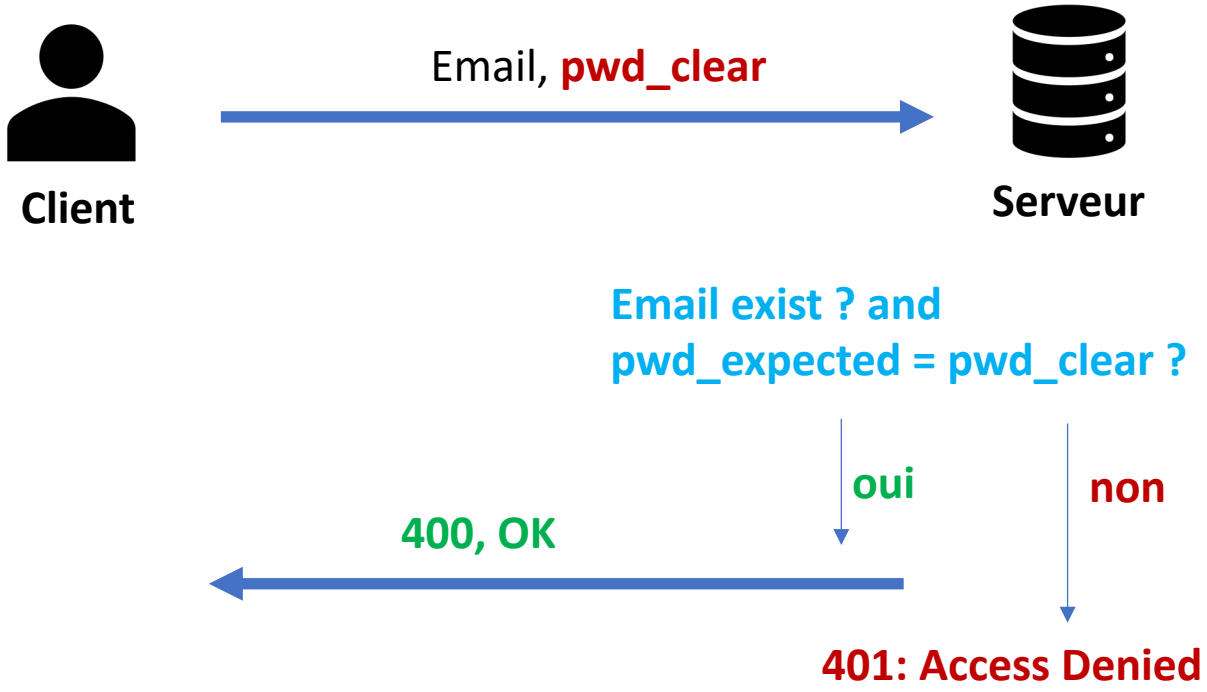
- Controller, service, repository
- Entity, exception

Gestion des exceptions obligatoire

- Créer au minimum ces exceptions personnalisées :
- InvalidInputException
- AuthenticationFailedException
- ResourceConflictException

Mettre en place un @ControllerAdvice pour renvoyer des réponses JSON cohérentes, avec :

- Timestamp, status, error
- Message, path



Codes HTTP attendus :

- 400 si données invalides
- 401 si login échoue
- 409 si email déjà existant

Qualité logicielle obligatoire

Tests unitaires attendus :

- Junit Minimum 8 tests.
- Validation email (vide, format incorrect).
- Validation mot de passe (moins de 4 caractères).
- Inscription OK.
- Inscription refusée si email déjà existant.
- Login OK.
- Login KO si mot de passe incorrect.
- Login KO si email inconnu.
- Accès /api/me : refus sans authentification, acceptation après login.

Documentation JavaDoc

- JavaDoc obligatoire sur :
- User (entité).
- AuthService (service principal).
- AuthController (endpoints).
- Exceptions.
- Le JavaDoc doit contenir une phrase explicite du type :
- Cette implémentation est volontairement dangereuse et ne doit jamais être utilisée en production.

Logging dans un fichier texte

- Logger les événements suivants :
- Inscription réussie.
- Inscription échouée.
- Connexion réussie.
- Connexion échouée.
- Interdiction :
- Ne jamais logger le mot de passe.

README obligatoire

- Comment lancer MySQL et configurer application.properties.
- Comment lancer l'API.
- Comment lancer le client Java.
- Le compte de test.
- Une section Analyse de sécurité TP1 avec au moins 5 risques majeurs expliqués simplement.

Git et étapes de versionnement obligatoires

L'objectif est que le Git raconte l'évolution du TP, pas un seul gros push final. **Étapes Git obligatoires à respecter**

Étape 0 : initialisation

- Créer le repository GitHub.
- Premier commit : projet vide Spring Boot + structure packages + README squelette.
- Tag : v1.0-init

Étape 1 : modèle de données

- Ajouter entité User + migration SQL ou script schema.sql.
- Ajouter repository.
- Commit propre.
- Tag : v1.1-model

Étape 2 : inscription

- Implémenter /api/auth/register.
- Validation minimale côté serveur.
- Gestion exceptions + ControllerAdvice.
- Tests associés.
- Tag : v1.2-register

Étape 3 : connexion

- Implémenter /api/auth/login.
- Gestion erreurs + logging.
- Tests associés.
- Tag : v1.3-login

Étape 4 : route protégée

- Implémenter /api/me.
- Mécanisme simple d'authentification (session ou token basique).
- Tests associés.
- Tag : v1.4-protected

Étape 5 : finalisation qualité

- Compléter JavaDoc.
- Nettoyer le code, supprimer warnings inutiles.
- Vérifier que tous les tests passent.
- Tag final : v1-tp1

Création rapide du projet

- Via https://start.spring.io

Project: Maven

- Language: Java
- Spring Boot: 3.x
- Java: 17

Dependencies :

- Spring Web
- Spring Data JPA
- H2 Database (pour démarrer vite)
- Lombok (optionnel)
- Spring Boot Test

yaml

```yaml
server:
  port: 8080

spring:
  datasource:
    url: jdbc:h2:mem:testdb
    driverClassName: org.h2.Driver
    username: sa
    password:

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true

  h2:
    console:
      enabled: true
```

Accès console H2 :

http://localhost:8080/h2-console

Structure recommandée des packages

- com.example.auth
- controller
- service
- repository
- entity
- exception

Entité User (TP1 – mot de passe en clair)

```java
import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)
    private String email;

    // TP1 volontairement dangereux
    private String password;

    private LocalDateTime createdAt;

    public User() {}

    public User(String email, String password) {
        this.email = email;
        this.password = password;
        this.createdAt = LocalDateTime.now();
    }

    // getters & setters
}
```

Repo & Services

5. Repository

```java
package com.example.auth.repository;

import com.example.auth.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
}
```

```java
package com.example.auth.service;

import com.example.auth.entity.User;
import com.example.auth.repository.UserRepository;
import org.springframework.stereotype.Service;

@Service
public class AuthService {

    private final UserRepository userRepository;

    public AuthService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User register(String email, String password) {

        if (password.length() < 4) {
            throw new IllegalArgumentException("Password too short");
        }

        if (userRepository.findByEmail(email).isPresent()) {
            throw new IllegalStateException("Email already exists");
        }

        User user = new User(email, password);
        return userRepository.save(user);
    }

    public boolean login(String email, String password) {
        return userRepository.findByEmail(email)
                .map(user -> user.getPassword().equals(password))
                .orElse(false);
    }
}
```

Controller REST

```java
package com.example.auth.controller;

import com.example.auth.service.AuthService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/register")
    public String register(@RequestParam String email,
                           @RequestParam String password) {

        authService.register(email, password);
        return "User registered";
    }

    @PostMapping("/login")
    public String login(@RequestParam String email,
                        @RequestParam String password) {

        boolean success = authService.login(email, password);

        if (!success) {
            return "Login failed";
        }

        return "Login success";
    }
}
```

Test Junit Simple

```java
package com.example.auth;

import com.example.auth.service.AuthService;
import com.example.auth.repository.UserRepository;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class AuthServiceTest {

    @Autowired
    private AuthService authService;

    @Test
    void testRegisterAndLogin() {
        authService.register("test@example.com", "abcd");
        boolean result = authService.login("test@example.com", "abcd");
        assertTrue(result);
    }
}
```

Lancer l'application (Maven) : -> mvn spring-boot:run

Tester avec Postman :

- POST http://localhost:8080/api/auth/register
- POST http://localhost:8080/api/auth/login

TP2 – Authentification Fragile

Objectif:

- Améliorer l'authentification du TP1 en ajoutant une politique de mot de passe stricte, un stockage serveur correct (hash adaptatif) et un verrouillage anti brute force
- Une première démarche qualité outillée avec **SonarCloud**.
- Malgré tout, l'authentification reste fragile, car le secret circule encore dans la phase de login et **reste rejouable** si une requête est capturée.
- TP3 et TP4 corrigeront cela via une **clé secrète partagée** et un **protocole anti-rejeu**.
- Durée : 10 heures (**1 semaine**)
- Tag final attendu : **v2-tp2**

Ajout majeur par rapport à votre version précédente

- **SonarCloud est obligatoire dès TP**2.
- On vise un premier niveau “qualité industrielle” avant même d’attaquer la crypto du TP3.

Fonctionnalités inchangé (rappel)

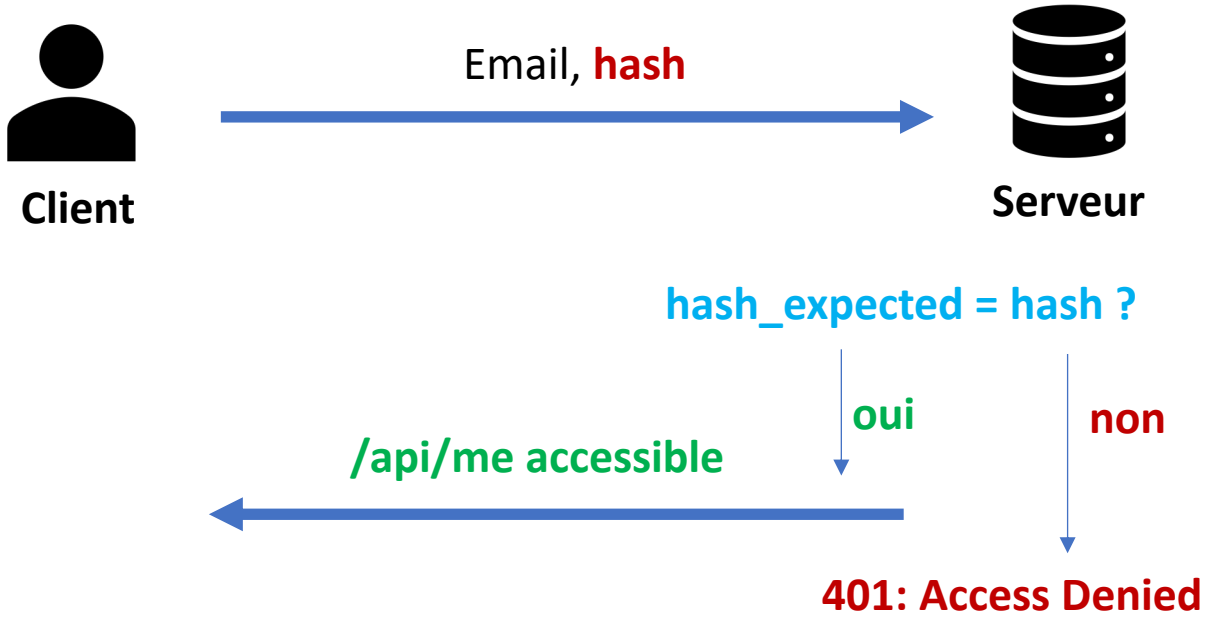
- 12 caractères min
- 1 majuscule, 1 minuscule, 1 chiffre, 1 caractère spécial.

Les nouvelles fonctionnalités attendues côté client

- Double saisie avant inscription
- password et passwordConfirm côté client.

Indicateur de force côté client

- **Rouge** non conforme.
- **Orange** conforme mais faible.
- **Vert** conforme et bon niveau.



Les nouvelles fonctionnalités attendues côté serveur

- Hash adaptatif côté serveur (Bcrypt)
- Stockage en base password_hash.
- Anti brute force: 5 échecs puis blocage 2 minutes.
- Code HTTP 423 ou 429, à justifier.

Route protégée

- GET /api/me accessible uniquement si authentifié.

Qualité logicielle obligatoire

JUnit

- Minimum 10 tests comme défini précédemment, plus 2 tests supplémentaires recommandés :
- Test sur la non-divulgation des erreurs (même message pour email inconnu ou mauvais mot de passe).
- Test sur le lockout qui expire correctement.

JavaDoc

- Obligatoire sur toutes classes security, validator, services et exceptions.
- Ajouter une phrase claire dans AuthService JavaDoc :
- TP2 améliore le stockage mais ne protège pas encore contre le rejeu.

Exceptions

- Exceptions personnalisées et @ControllerAdvice obligatoire, identique au TP1 mais enrichi si besoin.

Le projet doit être **obligatoirement** analysé sur SonarCloud.

Vous devez corriger, au minimum :

- Les bugs majeurs détectés.
- Les vulnérabilités majeures détectées.

Un ensemble de **code smells** significatifs, en priorité ceux qui touchent :

- Gestion des exceptions.
- Validation.
- Duplication.
- Complexité inutile.
- Seuil qualité minimal attendu

Quality Gate en succès, ou à défaut justification écrite dans README :

- Quelles règles bloquent.
- Pourquoi.
- Ce qui sera corrigé au TP4 avec CI/CD.

Couverture de tests

- À partir de TP2, vous commencez à mesurer la couverture.
- Objectif réaliste TP2 : 60 pour cent minimum.
- L'objectif 80 pour cent est imposé au TP3, quand le périmètre sera plus structuré.

Git et étapes de versionnement obligatoires

Étape 0 : démarrage TP2

- Mise à jour README avec objectifs TP2.
- Tag : v2.0-start

Étape 1 : migration base

- password_clear vers password_hash.
- Tag : v2.1-db-migration

Étape 2 : policy mot de passe

- PasswordPolicyValidator + tests.
- Tag : v2.2-password-policy

Étape 3 : hashing

- BCrypt PasswordEncoder.
- Inscription stocke hash, login vérifie.
- Tag : v2.3-hashing

Étape 4 : anti brute force

- failed attempts + lock_until + tests.
- Tag : v2.4-lockout

Étape 5 : UI force mot de passe

- Rouge orange vert côté client.
- Tag : v2.5-ui-strength

Étape 6 : SonarCloud

- Configurer SonarCloud, lancer analyse, corriger points majeurs.
- Ajouter section Qualité dans README avec capture ou résumé.
- Tag : v2.6-sonarcloud

Étape 7 : finalisation

- JavaDoc final, nettoyage, tous les tests verts.
- Tag final : v2-tp2

Pourquoi TP2 reste fragile:

- Même avec un hash en base, la phase de login repose encore sur une preuve directement dérivée de la saisie utilisateur.
- Si un attaquant capture la requête de login, il peut tenter de la rejouer.

Dans TP3, on ne se contentera pas de

- “mieux stocker”,
- On changera le protocole avec une clé secrète partagée, un nonce

README obligatoire,

- Section Qualité ajoutée
- Expliquer la faiblesse de l’authentification
- Comment SonarCloud est configuré.
- Résultat du Quality Gate.
- Couverture approximative et comment elle est mesurée.
- Plan d’améliorations futurs

TP3 – Authentification Forte

Objectif:

- Dans TP3, vous ne vous contentez plus d'améliorer le stockage du mot de passe mais vous changez le protocole d'authentification.
- Le mot de passe **ne doit plus être transmis** tel quel dans la requête de login (même en version haché).
- Le client doit prouver qu'il connaît un secret, sans l'envoyer directement.
- Durée : 10 heures (**1 semaine**)
- Tag final attendu : **v3-tp3**

Les nouvelles fonctionnalités

On introduit:

- Clé secrète partagée
- HMAC
- Nonce
- Timestamp
- Comparaison en temps constant
- Protection anti-rejeu

Conserver SonarCloud.

- Objectif couverture minimum : 80%

Concept clé pédagogique:

-> On passe de : “J’envoie un mot de passe” à

-> “Je **prouve** que je **connais** un secret sans l’envoyer”.

Principe global

Chaque utilisateur possède :

- Un secret partagé avec le serveur dérivé de son mot de passe
- Stocké côté serveur sous forme hashée ou clé dérivée

L’authentification devient un protocole en 2 étapes
Single Sign On (SSO) en un seul échange réseau

Le mot de passe ne circule jamais (en clair ou même hash sur le réseau)

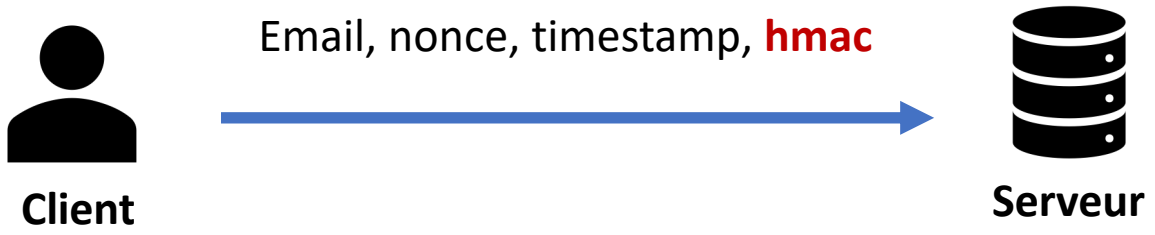
Hypothèse de travail:

1. **TLS est activé et fonctionnel. On ne le configure pas, on le suppose.**
2. **Le serveur stocke un mot de passe chiffré réversible en base.**
3. **Une Server Master Key (SMK) chiffre et déchiffre les mots de passe.**

SSO Etape 1: Le client prépare et envoie sa preuve d'identité

Le client collecte:

- Email, password (saisi par l'utilisateur)
- nonce (UUID aléatoire)
- timestamp (epoch, secondes)



Message à signer:

- **message = email + ":" + nonce + ":" + timestamp**

Signature:

- **hmac** = **HMAC_SHA256(key = password, data = message)**

Le client envoie:

- **POST /api/auth/login**

Payload JSON:

- Email, nonce, timestamp, hmac

Étape 1 côté client:

- **Calculer une preuve signée**
- **Envoie au serveur**

SSO Etape 2: Le Serveur vérifie l'identité

Vérifications obligatoires dans cet ordre

1. Vérifier email existe. Sinon 401.
2. Vérifier timestamp dans fenêtre. Exemple ± 60 secondes. Sinon 401.
3. Vérifier nonce anti-rejeu. Rejeter 401 si nonce déjà vu pour cet utilisateur.
4. Sinon réserver ou enregistrer nonce.
5. Récupérer mot de passe en clair .

Recalculer signature serveur.

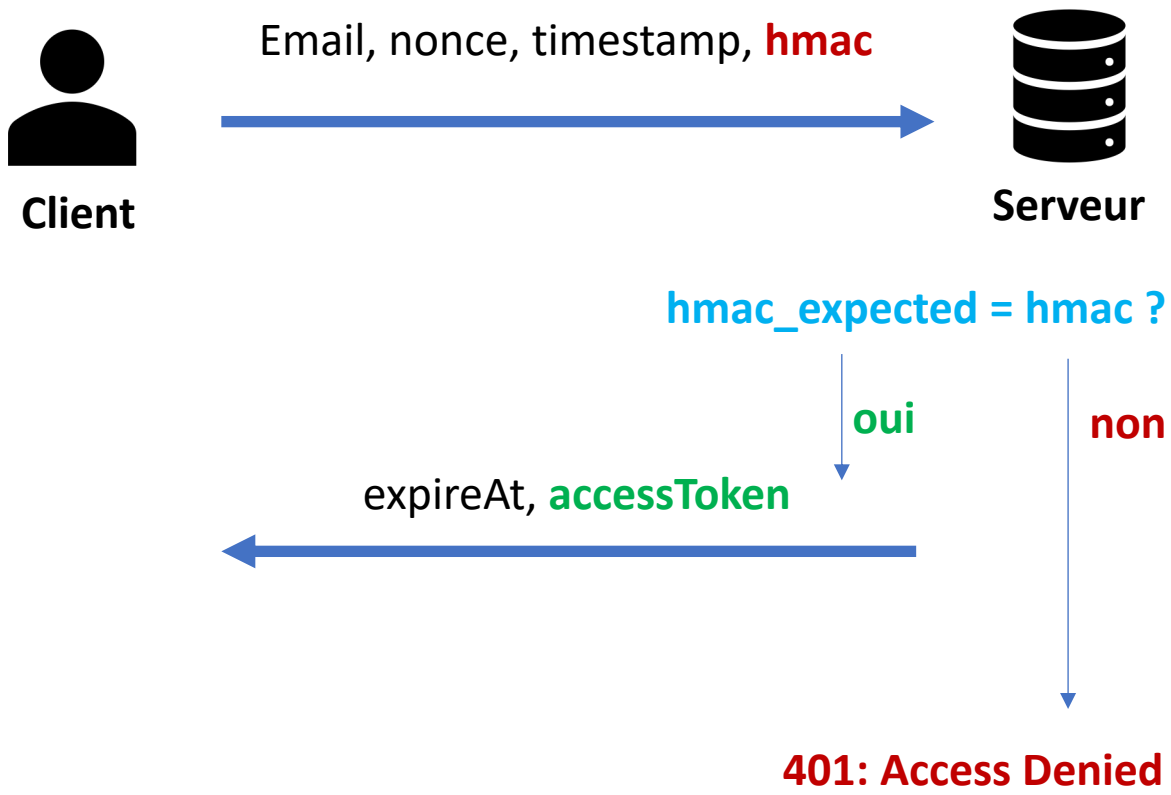
1. message = email + ":" + nonce + ":" + timestamp
2. hmac_expected = HMAC_SHA256(key = password_plain, data = message)
3. Comparer expected et hmac reçu en temps constant. Si différent 401.
4. Marquer nonce comme consommé et empêcher tout rejeu.

Émettre token SSO et retourner:

1. accessToken
2. expiresAt

A réception, le client affiche le accessToken dans la fenêtre

Étape 2 côté Serveur : vérifier la preuve reçue



Considérations techniques

Table users:

- Id, email
- password_encrypted, created_at

Table auth_nonce:

- Id, user_id
- Nonce, expires_at, consumed, created_at

Contraintes:

- (user_id, nonce) unique
- expires_at environ now + 2 minutes

Durées recommandées

- Fenêtre timestamp acceptée: 60 secondes.
- TTL nonce en base: 120 secondes.
- Access token: 15 minutes.

Exigences qualité TP3

- JUnit
- Minimum 15 tests.
- Couverture SonarCloud minimum 80 pour cent.

SonarCloud: Quality gate doit passer.

Tests obligatoires

- Login OK avec hmac valide.
- Login KO hmac invalide.
- KO timestamp expiré.
- KO timestamp futur.
- KO nonce déjà utilisé.
- KO user inconnu.
- Comparaison temps constant testée.
- Token émis et accès /api/me OK.
- Accès /api/me sans token KO.

Git Tags imposés

- v3.0-start
- v3.1-db-nonce
- v3.2-hmac-client
- v3.3-hmac-server
- v3.4-anti-replay
- v3.5-token
- v3.6-tests-80
- v3-tp3

JavaDoc: Documenter le protocole et les limites du chiffrement réversible

Avantages du protocole:

- Pas de mot-de-passe en clair ou haché qui circule sur le réseau
- Le timestamp limite la durée de l'attaque.
- Le nonce empêche la réutilisation.
- Sans stockage du nonce, le nonce ne sert à rien contre le rejeu.

Important à écrire dans le README

- Ce mécanisme est pédagogique.
- En industrie, on évite de stocker un mot de passe réversible.
- On préférerait un hash non réversible et adaptatif
- Ici, on accepte le chiffrement réversible pour simplifier l'apprentissage du protocole signé.

TP4 – Authentification et Master Key

Objectif:

On ne modifie plus le protocole HMAC, nonce, timestamp. On industrialise :

1. La protection des mots de passe au repos via une Master Key
2. L'automatisation complète via GitHub Actions
3. Le blocage des merges si qualité insuffisante

- Durée : 10 heures (**1 semaine**)
- Tag final attendu : **v4-tp4**

Contexte

- Dans TP3, les mots de passe étaient stockés en clair dans MySQL.
- Cela n'est plus acceptable.
- Désormais, une Master Key fournie par l'administrateur BP doit être utilisée pour chiffrer tous les mots de passe avant stockage.

Exigences obligatoires

- Le mot de passe ne doit jamais être stocké en clair.
- Le mot de passe doit être chiffré avant insertion en base.
- La Master Key ne doit jamais être dans le code.
- La Master Key doit être injectée via variable d'environnement.

Nom imposé : APP_MASTER_KEY

- Si la clé est absente, l'application doit refuser de démarrer.

Implémentation recommandée

- Algorithme : AES en mode GCM.
- AES GCM fournit chiffrement et intégrité.

Format de stockage conseillé :

- v1:Base64(iv):Base64(ciphertext)
- Structure MySQL mise à jour

Structure MySQL mise à jour

Table users :

- Id, email, password_encrypted, created_at
- Le champ password_clear est supprimé définitivement.

- **Processus à l'inscription :**
- password_plain
- → chiffrement AES via APP_MASTER_KEY
- → stockage password_encrypted

Processus au login :

- Lecture password_encrypted
- → déchiffrement
- → récupération password_plain
- → recalcul HMAC
- → vérification

Interdictions strictes

- Pas de clé codée en dur
- Pas d'IV fixe
- Pas de mode ECB
- Pas de log de mot de passe

Tests obligatoires liés à la Master Key

- Test démarrage KO si APP_MASTER_KEY absente
- Test encryption/decryption OK
- Test mot de passe chiffré ≠ mot de passe clair
- Test déchiffrement KO si ciphertext modifié

Partie 2 – GitHub Actions CI/CD obligatoire (cœur du TP4)

Objectif

- Automatiser totalement la qualité du projet.
- À partir de TP4, aucun code ne doit être validé sans passer par une pipeline automatique.

Chaque **push** et chaque **pull request** doit déclencher :

- **Build** automatique
- Exécution des tests JUnit
- Analyse SonarCloud

-> Blocage si échec

Exigences obligatoires

- Le repository doit contenir un workflow GitHub Actions dans
- .github/workflows/ci.yml

Déclencheurs obligatoires

- push sur la branche main
- pull_request vers la branche main
- Pipeline minimal attendu

La pipeline doit effectuer automatiquement :

- Récupération du code (checkout)
- Installation du JDK 17
- Build du projet (Maven ou Gradle accepté)
- Exécution des tests
- Analyse SonarCloud
- Échec automatique si tests échouent
- Échec automatique si SonarCloud Quality Gate échoue

Important

- La pipeline doit faire échouer le build si :
- Un seul test échoue
- Le **Quality Gate** SonarCloud est **rouge**
- Configurer le repo pour bloquer les merges si la CI échoue.

SonarCloud obligatoire

Le projet doit être connecté à SonarCloud.

- Configurer dans GitHub → Settings → Secrets :
- SONAR_TOKEN
- SONAR_PROJECT_KEY
- SONAR_ORGANIZATION
- La pipeline doit utiliser ces secrets.
- Le Quality Gate doit être en succès.

Gestion des tests en CI

- Les tests doivent pouvoir tourner sans base MySQL externe.
- Exigence obligatoire :
- Utiliser H2 en mémoire pour les tests.
- Cela garantit que la pipeline fonctionne sans dépendance externe.

Gestion de la Master Key en CI

- La Master Key réelle ne doit jamais être exposée.
- Pour les tests, utiliser une clé fictive injectée via :
- Variable d'environnement
- Profil application-test

Exemple :

- APP_MASTER_KEY=test_master_key_for_ci_only

Critères de succès:

- La pipeline s'exécute automatiquement
- Les tests sont réellement exécutés
- Le build échoue si un test échoue
- Le Quality Gate SonarCloud est vert
- Les secrets ne sont pas commités

```yaml
name: CI - Build, Test and SonarCloud

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      # 1. Récupération du code
      - name: Checkout repository
        uses: actions/checkout@v4

      # 2. Installation du JDK
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '17'
          cache: 'maven'

      # 3. Définition variable Master Key pour tests
      - name: Set test Master Key
        run: echo "APP_MASTER_KEY=test_master_key_ci" >> $GITHUB_ENV

      # 4. Build + Tests
      - name: Build and run tests
        run: mvn clean verify

      # 5. Analyse SonarCloud
      - name: SonarCloud Scan
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
        run: |
          mvn sonar:sonar \
            -Dsonar.projectKey=${{ secrets.SONAR_PROJECT_KEY }} \
            -Dsonar.organization=${{ secrets.SONAR_ORGANIZATION }} \
            -Dsonar.host.url=https://sonarcloud.io
```

Exemple fichier: .github/workflows/ci.yml

Ce que ce fichier fait

- Déclenchement automatique sur push et pull request vers main
- Installe Java 17
- Met en cache Maven
- Injecte une Master Key fictive pour les tests
- Exécute tous les tests
- Lance SonarCloud
- Échoue si tests échouent
- Échoue si SonarCloud Quality Gate échoue

A configurer dans GitHub:

- Dans Settings → Secrets and variables → Actions

Ajouter :

- SONAR_TOKEN
- SONAR_PROJECT_KEY
- SONAR_ORGANIZATION